Aula 4 – Listas

Listas Simplesmente Ligadas

Listas Duplamente Ligadas

Exemplos

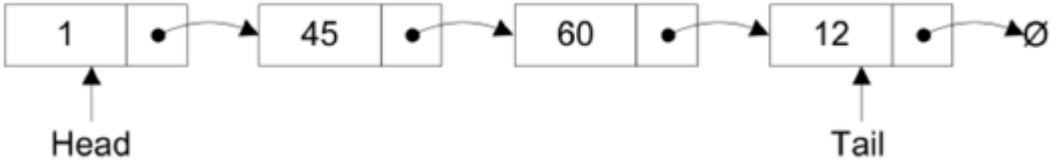
Listas Ligadas

Inserção O(1) - Remoção O(n) - Pesquisa O(n)

Métodos:

- void addFirst(Object o); // adiciona no inicio
- void addLast(Object o); // adiciona no fim
- boolean contains(Object o); // se contem um objecto
- boolean remove(Object o); // remove um objecto
- Object peekFirst(); // Devolve o primeiro objecto da lista
- Object peekLast(); // Devolve o último objecto da lista

Listas Simplesmente Ligadas

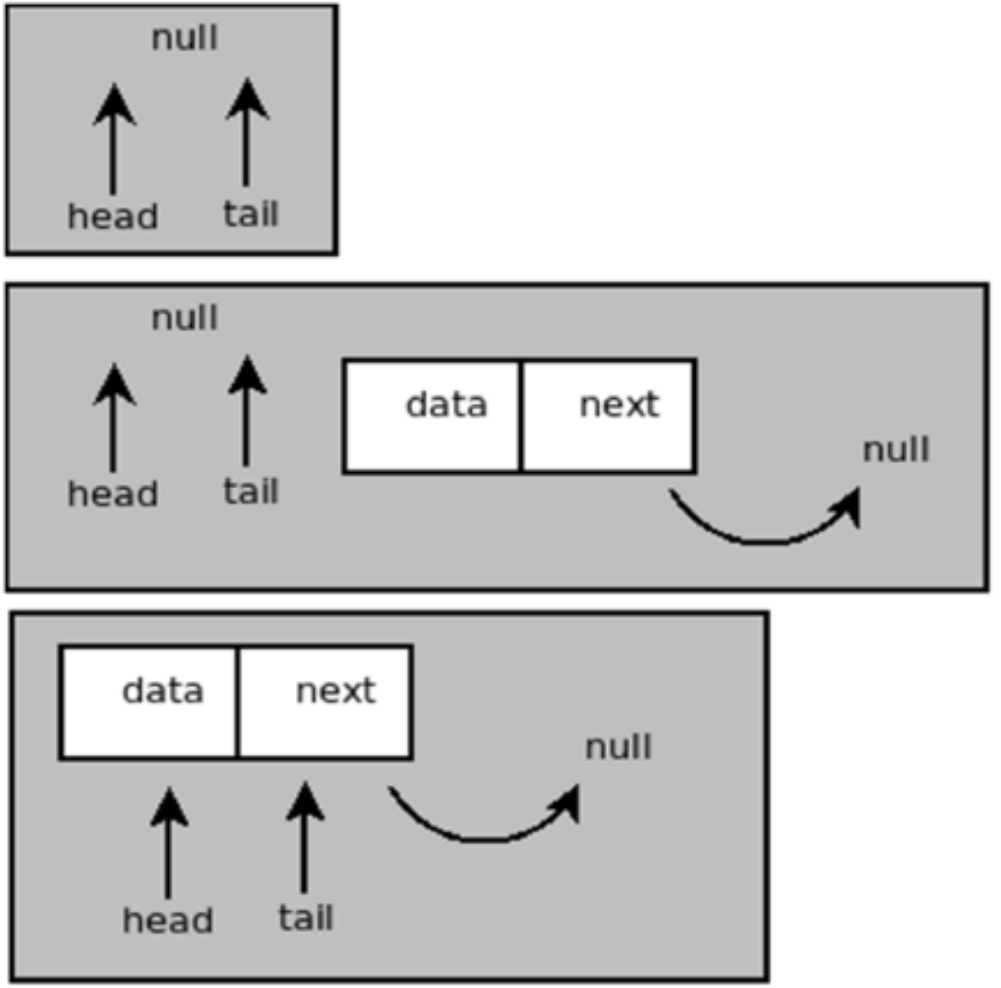



```
Item {
Object data
Item next
}
```

```
List {
    Item head
    Item tail
}
```

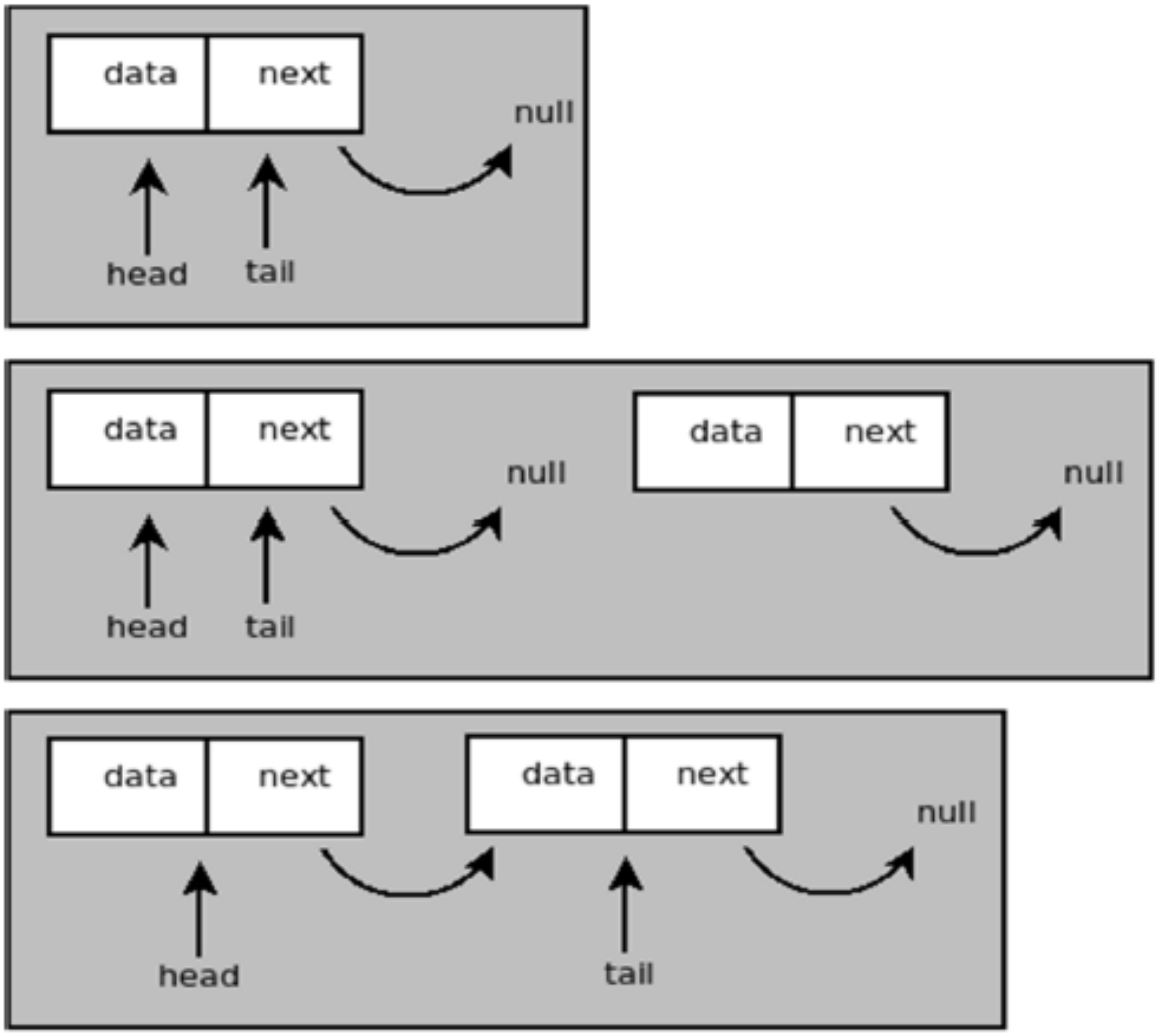
Listas Simplesmente Ligadas

```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    Se head = null
        head = item
        tail = item
    Se não
        tail.next = item
        tail = item
}
```



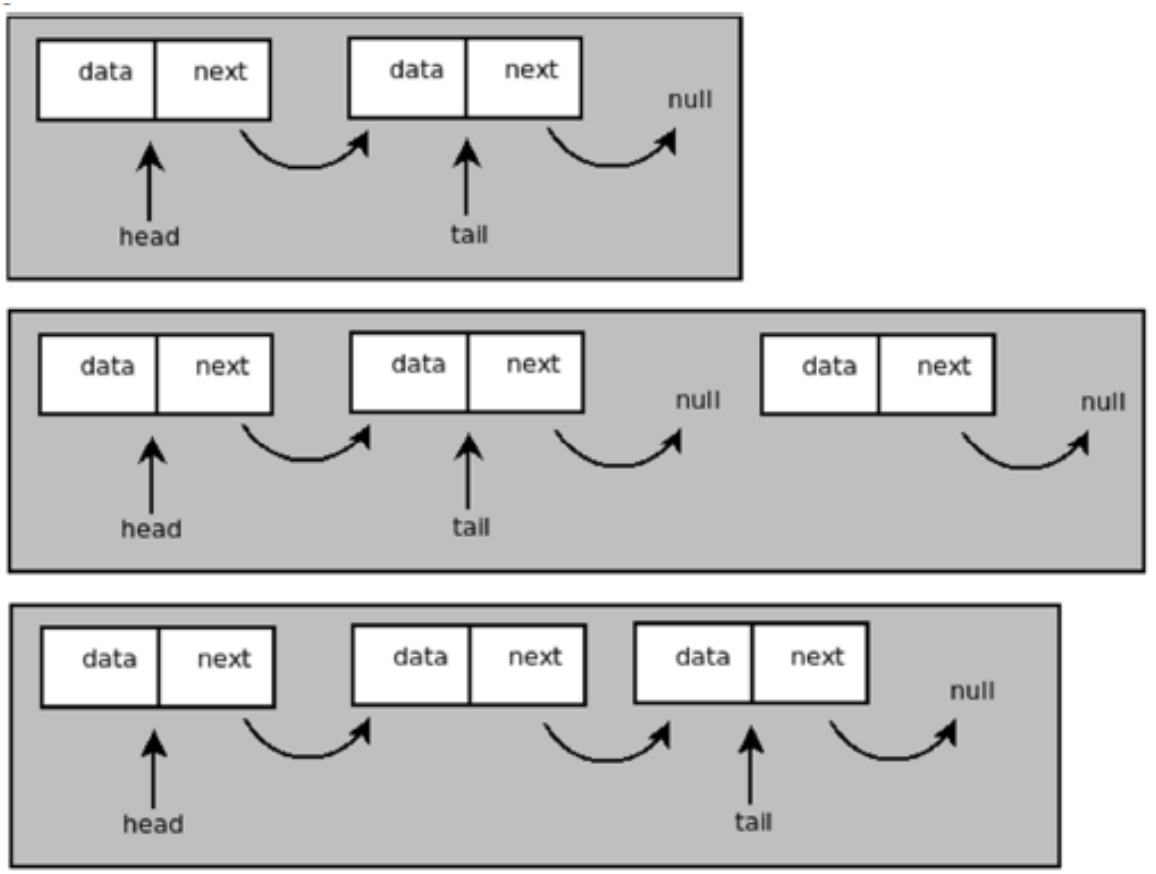
Listas Simplesmente Ligadas

```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    Se head = null
        head = item
        tail = item
    Se não
        tail.next = item
        tail = item
}
```



Listas Simplesmente Ligadas

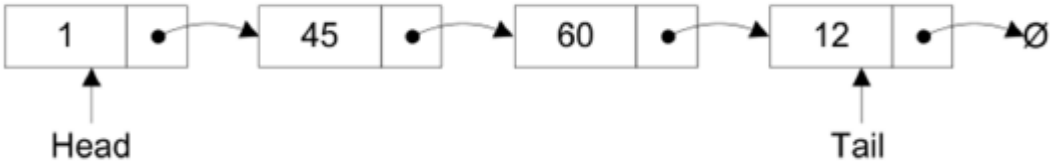
```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    Se head = null
        head = item
        tail = item
    Se não
        tail.next = item
        tail = item
}
```



Listas Simplesmente Ligadas

```
void addFirst(Object data) {
    novo(item)
    item.data = data
    item.next = null
    Se head = null
        head = item
        tail = item
    Se não
        item.next = head
        head = item
}
```

Listas Simplesmente Ligadas




```
boolean contains(Object data) {
    i = head
    Enquanto i <> null e i.data <> data
        i = i.next
    Se i = null
        Devolve falso
    Se não
        Devolve verdadeiro
}
```

Listas Simplesmente Ligadas

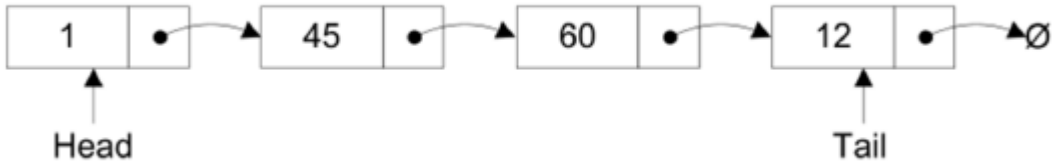
```
boolean remove(Object data) {

 }
```

É necessário prever 6 situações:

1. A lista está vazia
2. O item a remover é o único da lista
3. O item a remover é a cabeça
4. O item a remover é a cauda
5. O item a remover está no meio da lista
6. O item não se encontra na lista

Listas Simplesmente Ligadas




```
boolean remove(Object data) {
    Se head = null  // Situação 1
       Devolve falso
    Se não
        i = head
        Se i.data = data
            Se head = tail // Situação 2
                head = null
                tail = null
            Se não // Situação 3
                head = head.next
            Devolve verdadeiro
        Enquanto i.next <> null e i.next.data <> data
            i = i.next
        Se i.next <> null
            Se i.next = tail // Situação 4
                tail = i
            i.next = i.next.next // Situação 5
            Devolve verdadeiro
        Devolve falso // Situação 6
}
```

Listas Simplesmente Ligadas

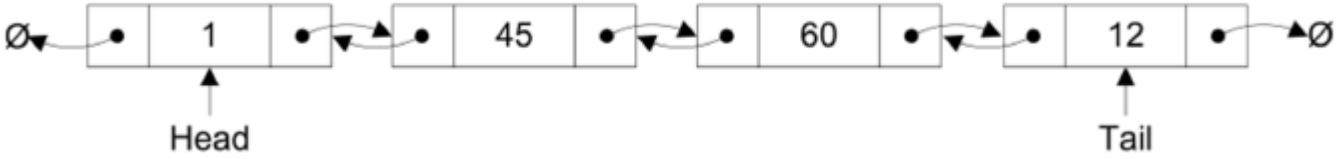
Percorrer uma LSL do inicio para o fim:

```
i = head
Enquanto i <> null
    // tratar i.data
    i= i.next
```

Percorrer um LSL do fim para o inicio:

```
Se tail <> null
    i = tail
    Enquanto i <> head
        anterior = head
        Enquanto anterior.next <> i
            anterior = anterior.next
        // tratar i.data
        i = anterior
    // tratar i.data
```

Listas Duplamente Ligadas

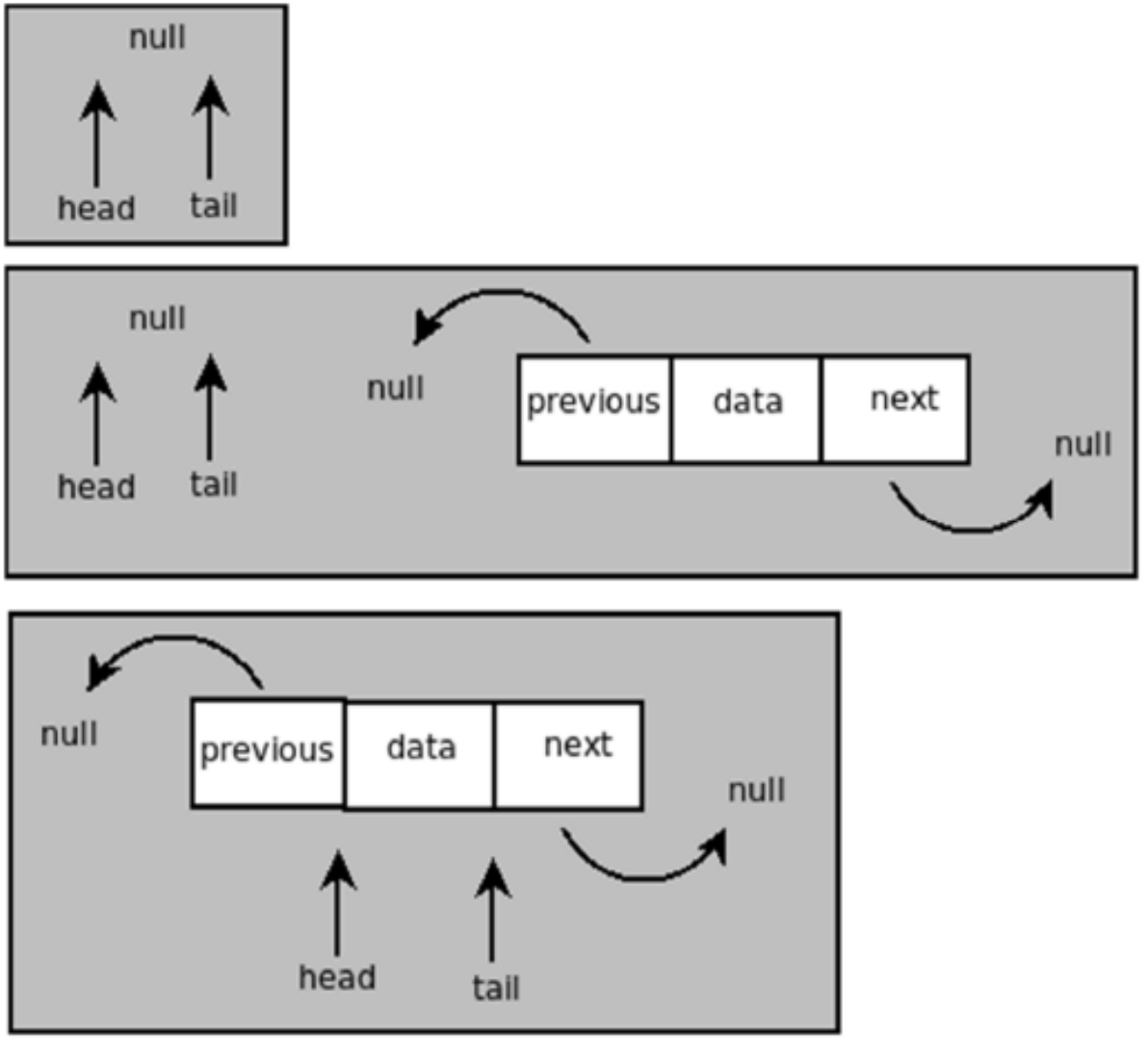



```
Item {
Object data
Item next
Item previous
}
```

```
List {
    Item head
    Item tail
}
```

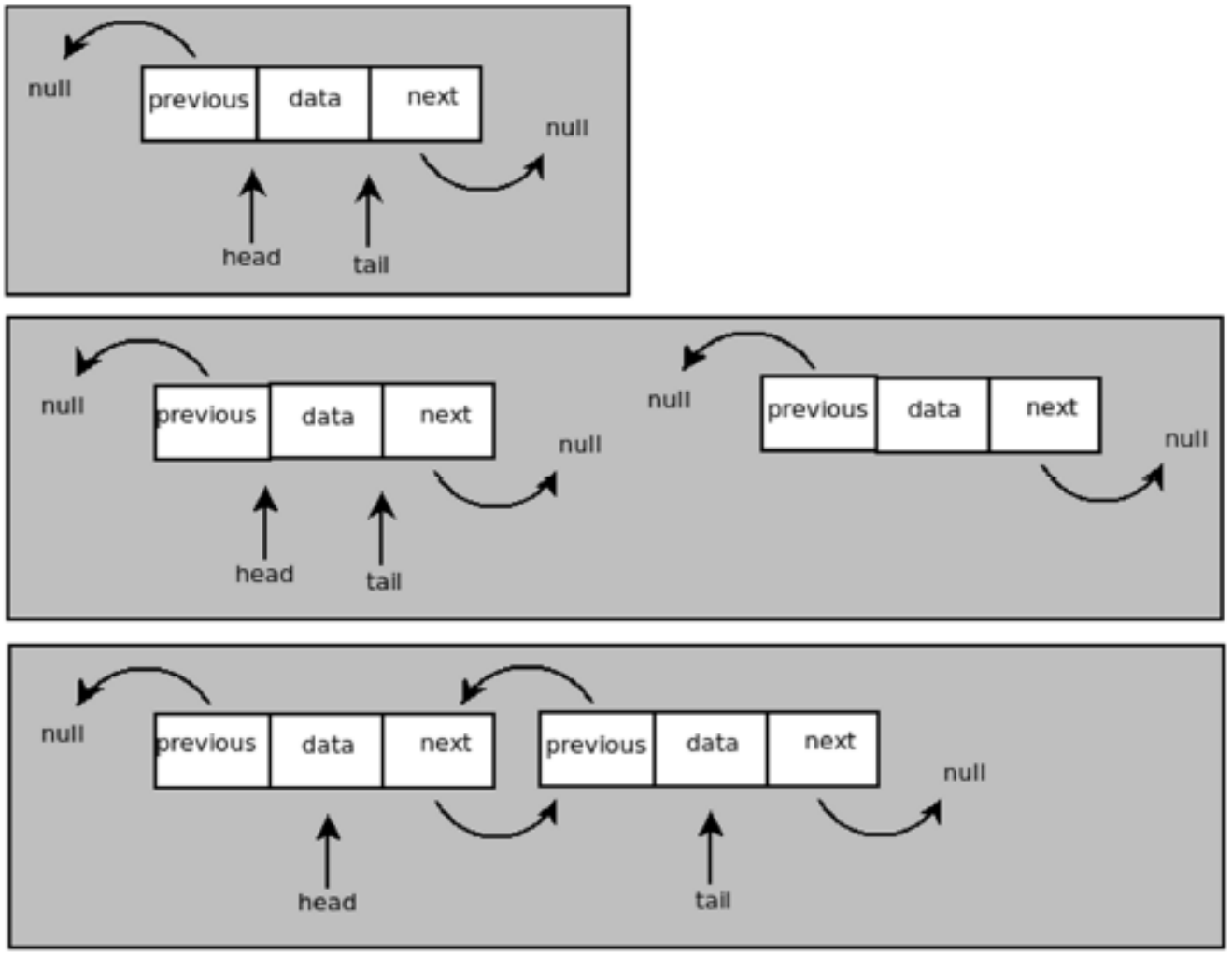
Listas Duplamente Ligadas

```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    item.previous = null
    Se head = null
        head = item
        tail = item
    Se não
        item.previous = tail
        tail.next = item
        tail = item
}
```



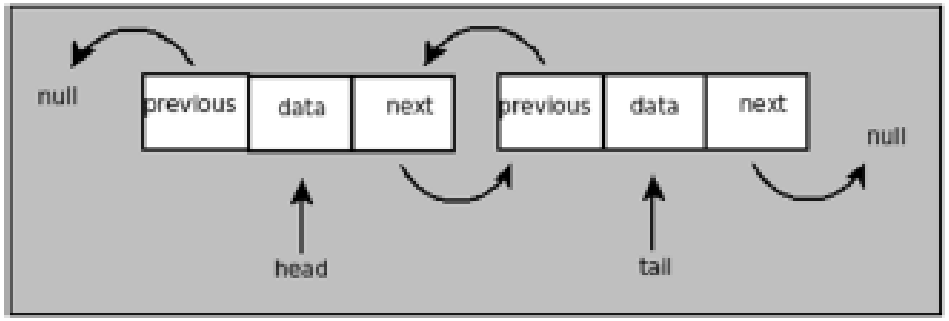
Listas Duplamente Ligadas

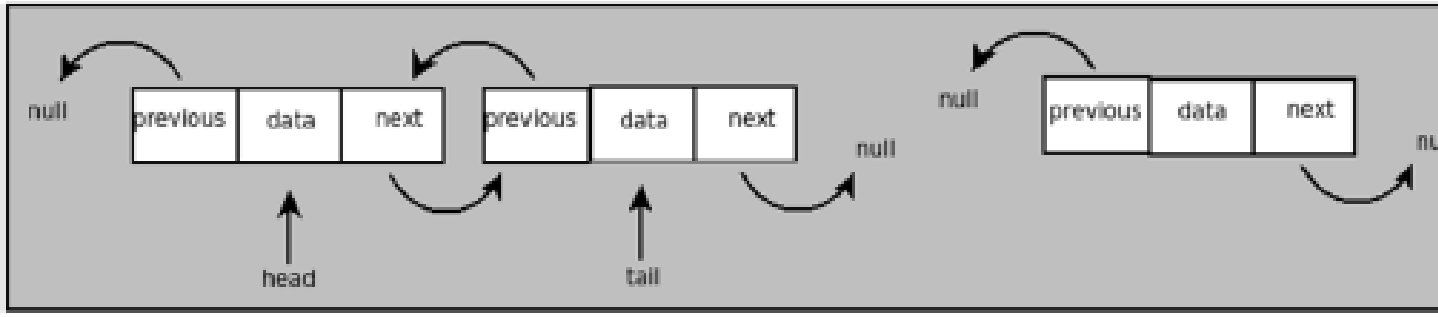
```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    item.previous = null
    Se head = null
        head = item
        tail = item
    Se não
        item.previous = tail
        tail.next = item
        tail = item
}
```

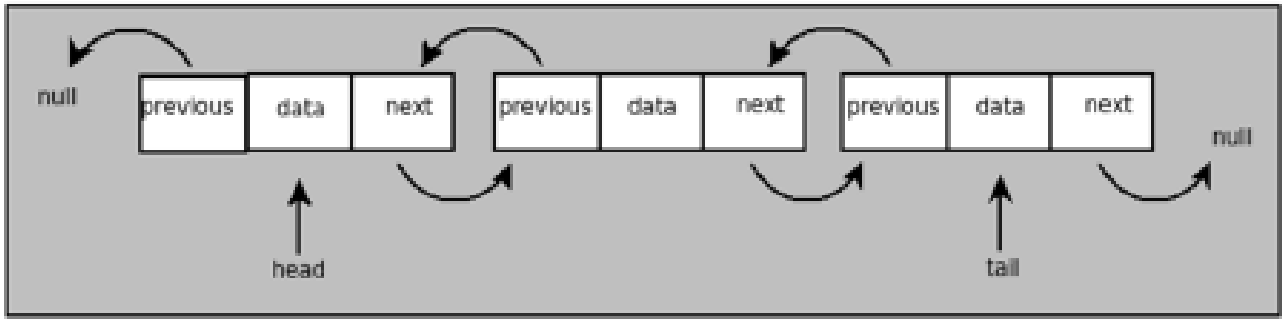


Listas Duplamente Ligadas

```
void addLast(Object data) {
    novo(item)
    item.data = data
    item.next = null
    item.previous = null
    Se head = null
        head = item
        tail = item
    Se não
        item.previous = tail
        tail.next = item
        tail = item
}
```



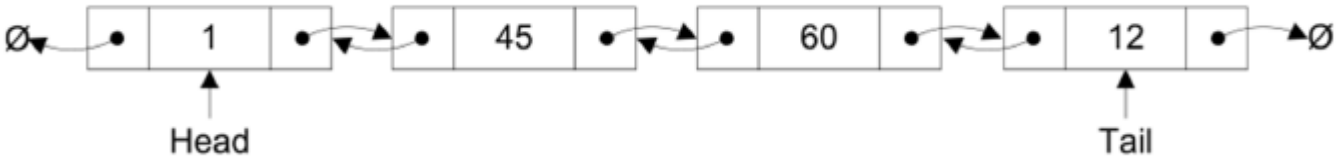




Listas Duplamente Ligadas

```
void addFirst(Object data) {
    novo(item)
    item.data = data
    item.next = null
    item.previous = null
    Se head = null
        head = item
        tail = item
    Se não
        item.next = head
        head.previous = item
        head = item
}
```

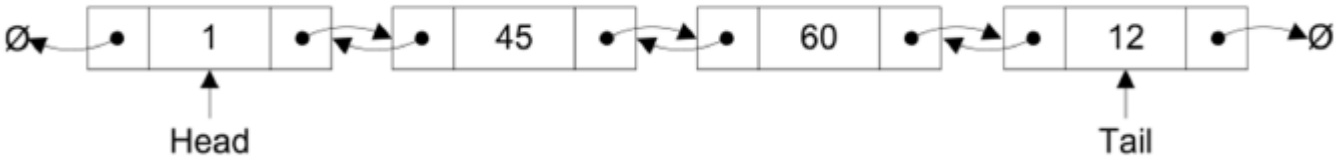
Listas Duplamente Ligadas




```
boolean contains(Object data) {
    i = head
    Enquanto i <> null e i.data <> data
        i = i.next
    Se i = null
        Devolve falso
    Se não
        Devolve verdadeiro
}
```

```
boolean contains(Object data) {
    i = tail
    Enquanto i <> null e i.data <> data
        i = i.previous
    Se i = null
        Devolve falso
    Se não
        Devolve verdadeiro
}
```

Listas Duplamente Ligadas




```
boolean remove(Object data) {
    Se head = null
       Devolve falso
    Se não
        Se data = head.data
            Se head = tail
                head = null
                tail = null
            Se não
                head = head.next
                head.previous = null
            Devolve verdadeiro
        i = head.next
        Enquanto i <> null e data <> i.data
            i = i.next
        Se i = tail
            tail = tail.previous
            tail.next = null
            Devolve verdadeiro
        Se não
            Se i <> null
                i.previous.next = i.next
                i.next.previous = i.previous
               Devolve verdadeiro
        Devolve falso
}
```

Listas Duplamente Ligadas

Percorrer uma LDL do inicio para o fim:

```
i = head

Enquanto i <> null

    // tratar i.data

    i= i.next
```

Percorrer um LDL do fim para o inicio:

```
i = tail

Enquanto i <> null

    // tratar i.data

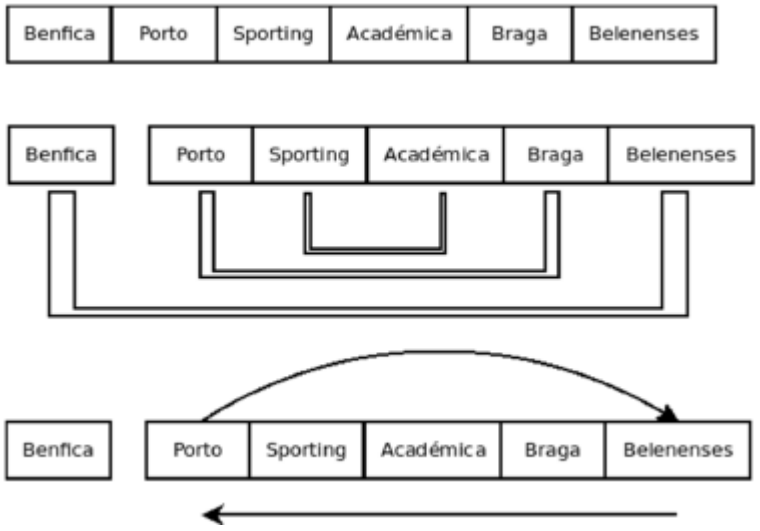
    i= i.previous
```

Listas Ligadas

Outros Métodos que também podem ser implementados:

- void insertSort(Object o); // Insere de forma ordenada
- int size(); // Devolve o número de elementos
- Object get(int i); // Devolve o objecto da posição i
- void addNext(int i, Object o); // insere o object a seguir à posicao i
- void addPrevious(int i, Object o); // insere o object antes da posicao i

Exemplos – Agendar Jogos



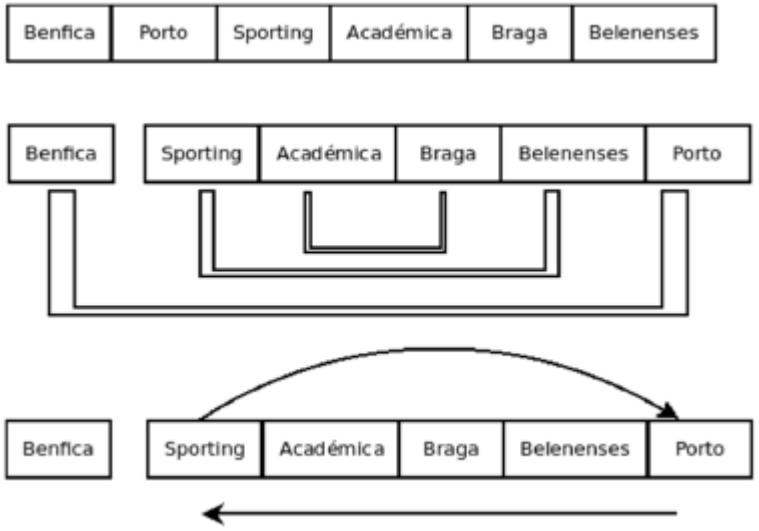
1ª Jornada:

Benfica – Belenenses

Porto – Braga

Sporting - Académica

Exemplos – Agendar Jogos



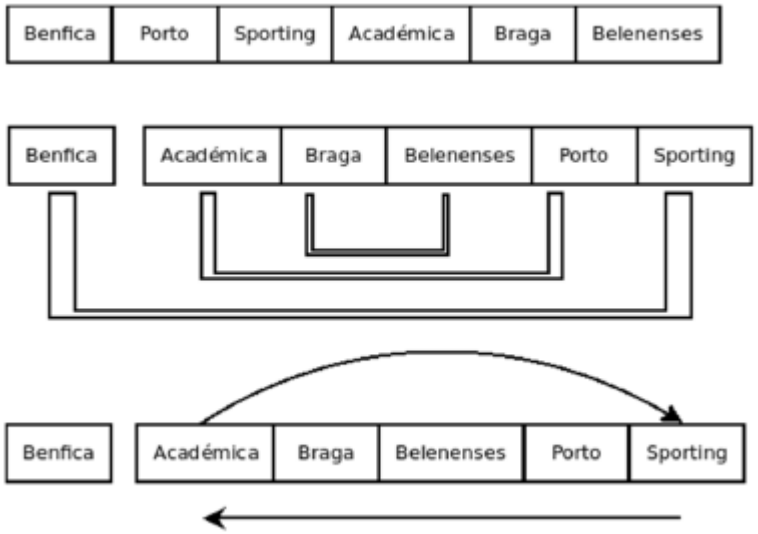
2ª Jornada:

Benfica – Porto

Sporting – Belenenses

Académica - Braga

Exemplos – Agendar Jogos



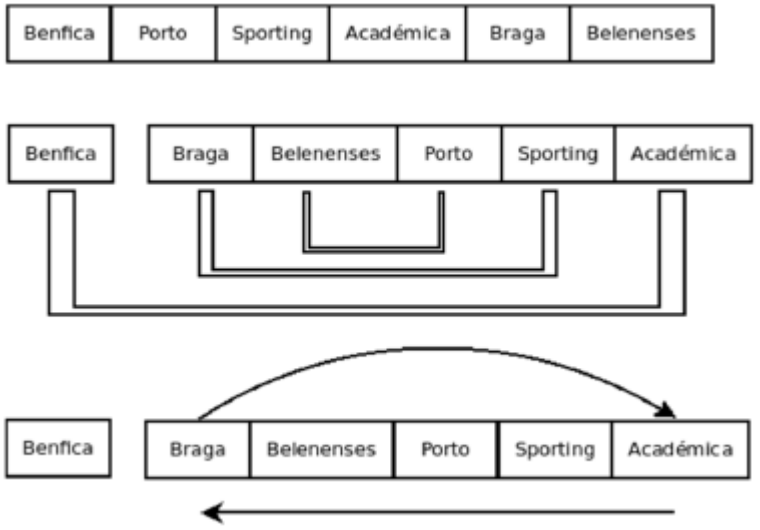
3ª Jornada:

Benfica – Sporting

Académica – Porto

Braga - Belenenses

Exemplos – Agendar Jogos



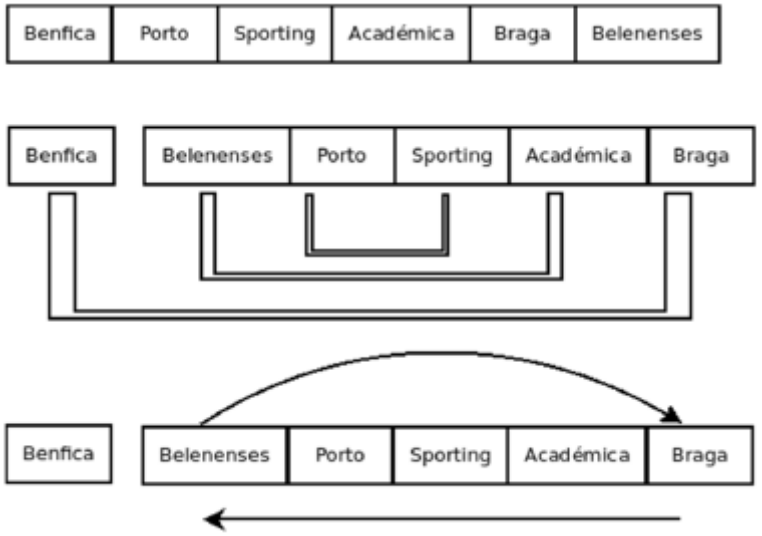
4ª Jornada:

Benfica – Académica

Braga – Sporting

Belenenses - Porto

Exemplos – Agendar Jogos



5ª Jornada:

Benfica – Braga

Belenenses – Académica

Porto - Sporting

Exemplos – Alocação de Memória

O Sistema Operativo, possui uma lista ligada para fazer a gestão de memória. Na lista cada item possui:

- Se o segmento de memoria está livre ou ocupado
- Inicio do Segmento
- Tamanho
- Próximo Segmento

Exemplos – Alocação de Memória

Algoritmo First Fit: Percorre a lista até encontrar o primeiro espaço disponível.

Algoritmo Next Fit: O mesmo que o anterior, só que em vez de partir sempre do inicio da lista, parte da última posição.

Algoritmo Best Fit: Verifica toda a lista para encontrar o segmento livre cujo tamanho melhor se aproxima às necessidades

Algortimo Worst Fit: Verifica toda a lista para encontrar o segmento livre com o maior tamanho.